

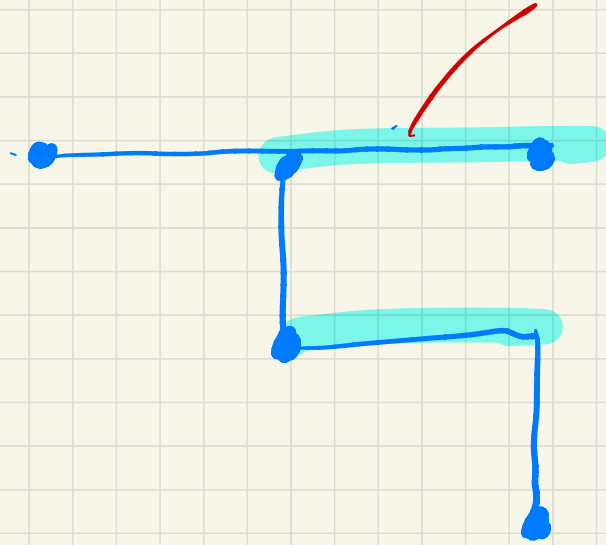
ECE 1387

Lecture #5

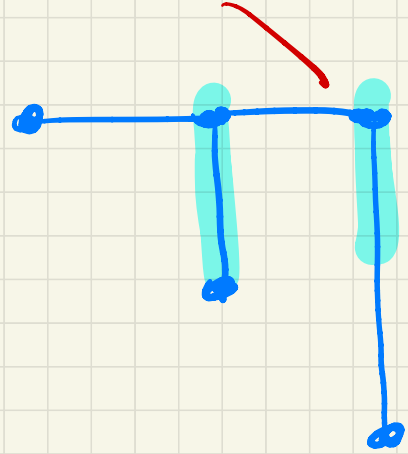
October 10, 2025

- A2 is almost posted
- Today: analytical placement (AP)

5-pin example:



● double
counted



HPWL is an
underestimate of WL for
high fanout nets.

Now we know a way to
estimate placement goodness

Many different placement algorithms
this course:

- analytical placement (today)
- Simulated-annealing

Analytical Placement (AP)

- widely used in commercial FPGA, ASIC placers
- Scalable to large designs
- papers from 90s, still active research to day
- Main idea:
 - "compute" a placement for all cells by solving a system of equations.

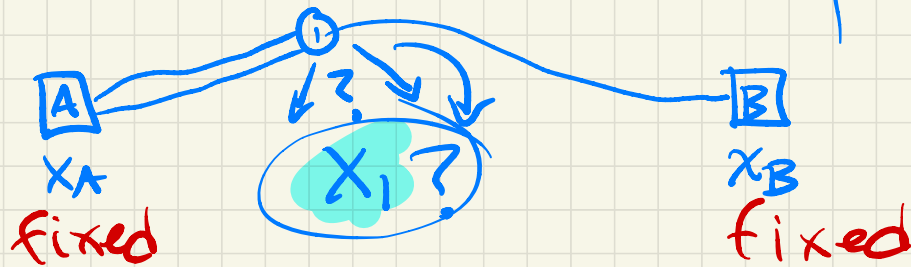
Most popular form:

Quadratic placement

Why?

Because it minimizes quadratic
WL (squared WL)

Consider 1D placement problem:

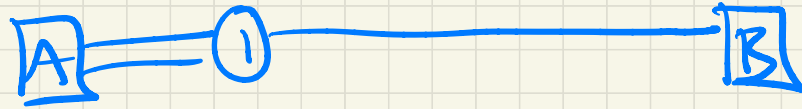


min linear WL

$$\min \left\{ |x_1 - x_A| + |x_1 - x_A| + |x_1 - x_B| \right\}$$

min quadratic WL

$$\min \{ (x_A - x_1)^2 + (x_A - x_1)^2 + (x_1 - x_B)^2 \}$$



↳ this is worse WL than minimizing linear!

So, why minimize quadratic WL?
— because we can easily!

AP inputs

- netlist of cells to place
- net model (later)

gives us

→ a weight w_{ij} representing
connectivity between cells i, j

Weight $w_{ij} = 0$ if cells i, j
are not connected to each other.

AP cost (objective) Function
minimize

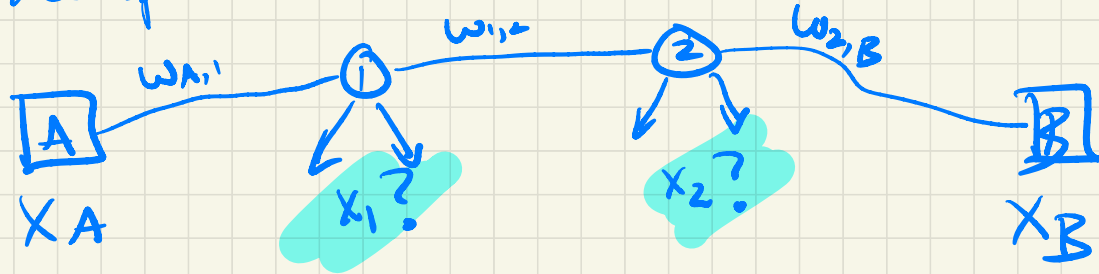
$$\Phi = \sum_{i \in \text{cells}} \sum_{j \in \text{cells}} w_{i,j} \cdot (x_i - x_j)^2 + w_{i,j} (y_i - y_j)^2$$

Can break into two probs

$$\min \Phi_x = \sum_{i \in \text{cell}} \sum_{j \in \text{cells}} w_{i,j} (x_i - x_j)^2 \quad \min \Phi_y = \dots$$

Can minimize independently

AP example formulation:



what we are solving for

$$\min \phi = w_{A,1} \cdot (x_A - x_1)^2 + w_{1,2} \cdot (x_1 - x_2)^2 + w_{2,B} \cdot (x_2 - x_B)^2$$

$$\min \phi = w_{A,1} \cdot (x_A - x_1)^2 + w_{1,2} \cdot (x_1 - x_2)^2 + w_{2,B} (x_2 - x_B)^2$$

$$\frac{\partial \phi}{\partial x_1} = -\cancel{2} w_{A,1} (x_A - x_1) + \cancel{2} w_{1,2} (x_1 - x_2) = 0$$

$$(w_{A,1} + w_{1,2}) x_1 + (-w_{1,2}) x_2 = w_{A,1} \cdot x_A$$

$$\frac{\partial \phi}{\partial x_2} = -\cancel{2} w_{1,2} (x_1 - x_2) + \cancel{2} w_{2,B} (x_2 - x_B) = 0$$

$$\frac{2\phi}{2x_2} = -\cancel{2}w_{1,2}(x_1 - x_2) + \cancel{2}w_{2,B}(x_2 - x_B) = \bar{c}$$

$$-w_{1,2}x_1 + (w_{1,2} + w_{2,B})x_2 = w_{2,B} \cdot x_B$$

In matrix form:

$$\begin{bmatrix} w_{A,1} + w_{1,2} & -w_{1,2} \\ -w_{1,2} & w_{1,2} + w_{2,B} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} w_{A,1} \cdot x_A \\ w_{2,B} \cdot x_B \end{bmatrix}$$

Matrix form

$$\mathbf{Q}_x \cdot \vec{x} = \vec{D}_x$$

— can solve
with standard
linear system
solver.

$n \times 1$ vectors // $n = \#$ moveable
cells

$n \times n$ matrix

$\mathbf{Q}_x$ is symmetric $\left\{ \begin{array}{l} \text{ } \\ \text{ } \end{array} \right. \Rightarrow$ sparse

$\vec{b}_x$ non-zero elements arise
from connections between
moveable & fixed cells

→ if $\vec{b}_x = 0$ then trivial
solⁿ $\vec{x} = 0$!

Need some fixed cells for
AP to work

Trick to formulate

$$Q\vec{x} = \vec{b}$$

$Q(i,i) =$ sum of all weights
on cell i

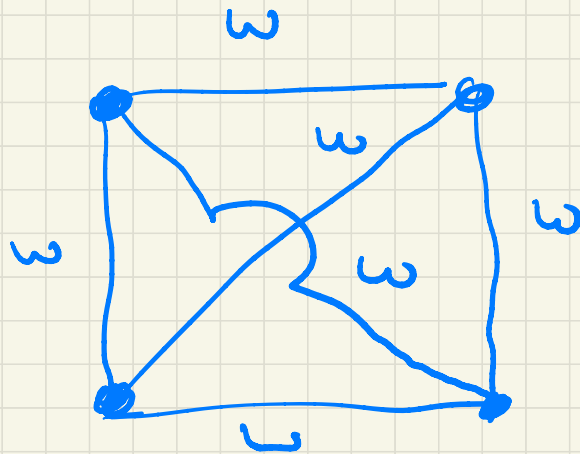
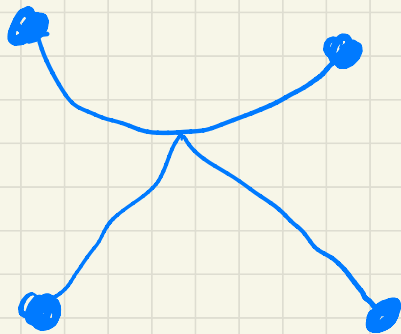
$$Q(i,j) = -w_{ij} \quad i \neq j$$

$b(i) =$ sum of all weights • Fixed
positions of cells
connected to i (not
include connections to movable
cells)

How to get edge weights?

w_{ij}

Easy way: clique model
say 4-pin net



Usually for a p -pin net
Set $w = \frac{2}{p}$

Rationale: p -clique has
 $\frac{p \cdot (p-1)}{2}$ edges

So, total weight = $\frac{p \cdot (p-1)}{2} \cdot \frac{2}{p}$

= $(p-1)$ # of edges
in a tree with p nodes

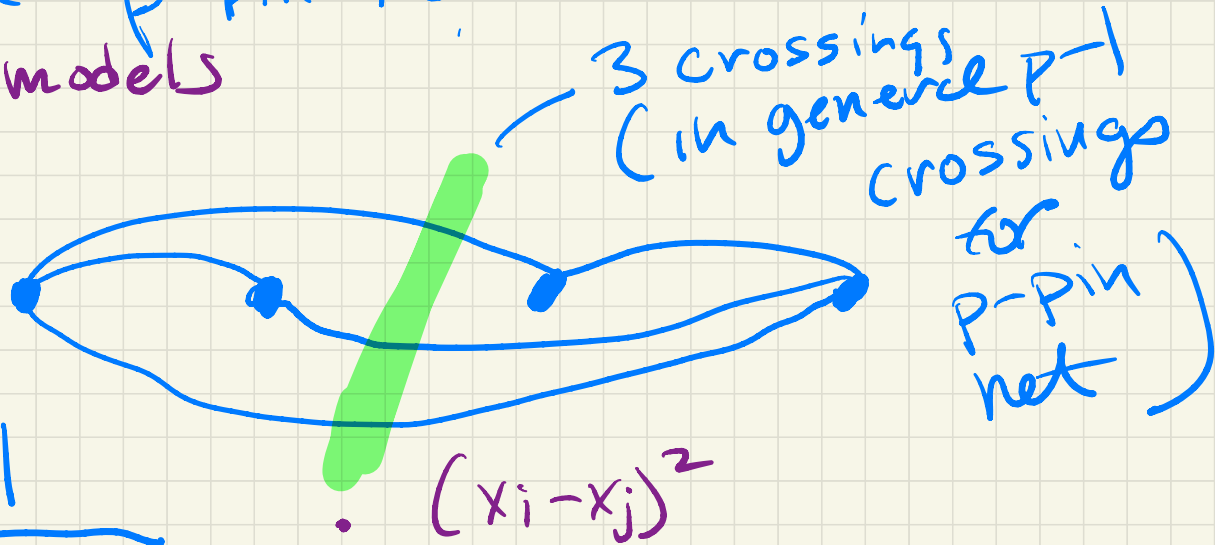
Bound-to-bound (B2B) model

- TU Munich 2008, widely used

- Given a p-pin net

- exactly models

HPWL



$$W_{xi,j} = \frac{1}{(p-1)|x_i - x_j|} \cdot (x_i - x_j)^2$$

Downside of B2B net model

- need a placement to compute weights
- weights are different in x, y dimensions

need to
iterate to
get
weights

x, y Placement

Adjust weights
using x_{old}, y_{old}

Solve for new
placement

x_{new}, y_{new}

Compare
 x_{new}, y_{new}
with
 x_{old}, y_{old}

are they
close?

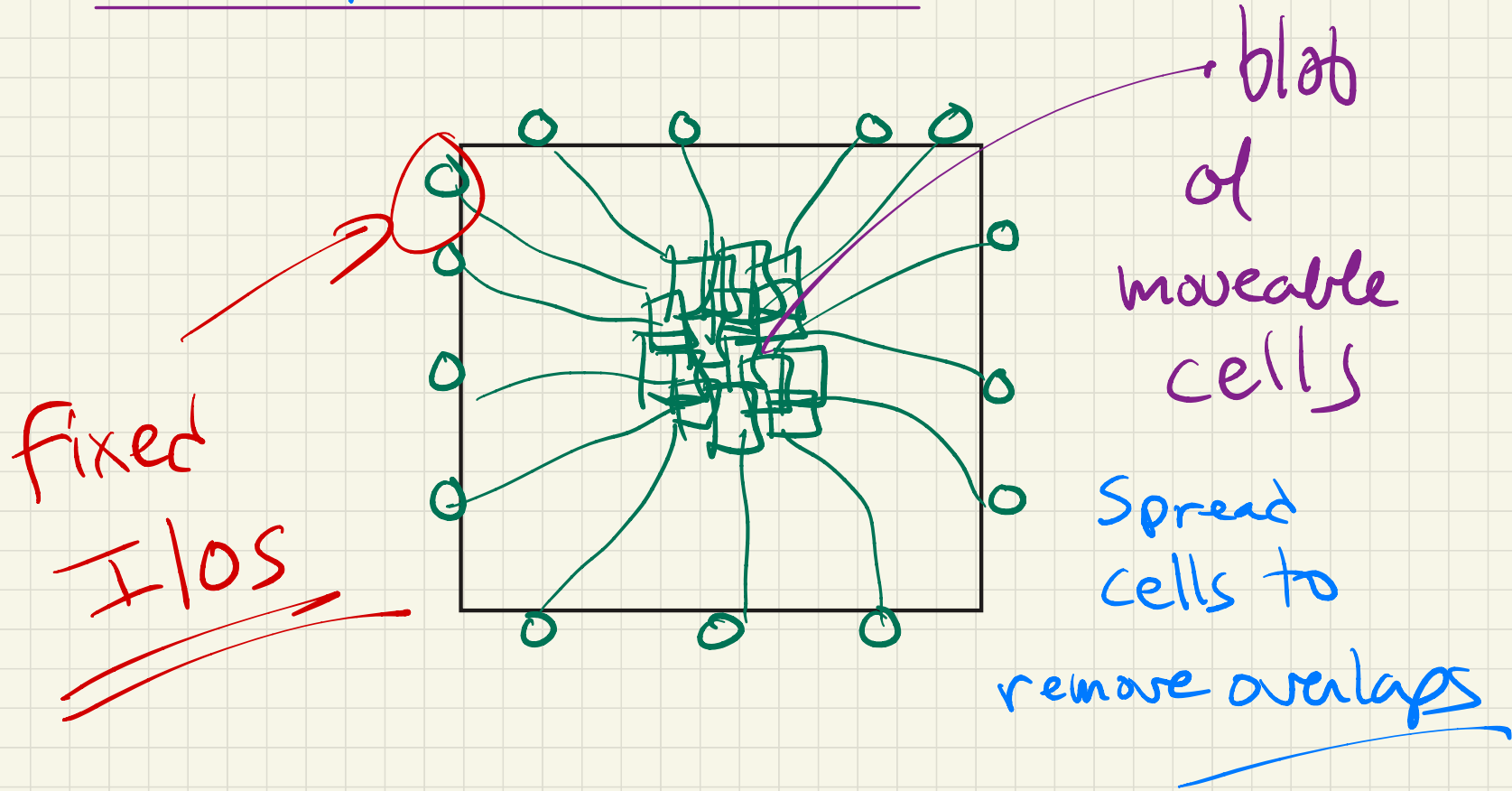
no

yes, close

found weights



AP output so far...



acm Association for Computing Machinery

Browse

Proceedings Books SIGs Conferences People

Search ACM Digital Library

Conference Proceedings Upcoming Events Authors Affiliations Award Winners

RESEARCH-ARTICLE

Multi-Commodity Flow-Based Spreading in a Commercial Analytic Placer

Authors:  [Nima Karimpour Darav](#),  [Andrew Kennings](#),  [Kristofer Vorwerk](#),  [Arun Kundu](#) [Authors Info & Claims](#)

FPGA '19: Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays • February 2019 • Pages 122–131 • <https://doi.org/10.1145/3289602.3293896>

Microchip/
Microsemi

Spreading
is how
AP algorithms

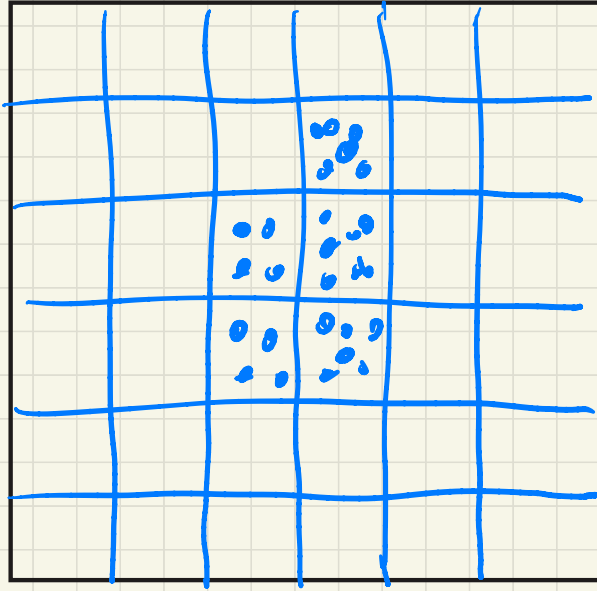
diffu

Best paper

Spreading

— overlay a "bin grid" on placement area

— want to
remove overlaps
with minimum
cell movement



for each overused bin b_i

- find selected paths to bins with capacity (underused)
- move cells along such paths
- cell movement can be at most ψ allowable cell movement
- ψ increases slowly

$$\text{Supply}(b_i) = \max(0, \text{usage}(b_i) - \text{capacity}(b_i))$$

$\text{Supply}(b_i) > 0$ means bin b_i
is overused.

Finding the "Spread"
placement (as
opposed to the "solved"
placement from linear system
solver)

iter = 0

repeat

$\psi = \max \text{ movement (iter)} // 2 \text{ iter}^2$

$B = \text{Set of over-filled bins}$
(sorted in ascending order
of supply)

for each bin $b_i \in B$ do

$P(b_i) = \text{List of candidate}$
paths for b_i

Sorted in ascending cost

```
for each  $P_k \in P(b_i)$  do  
  if  $\text{supply}(b_i) > 0$   
    move cells over  $P_k$   
  end if  
end for  
end for  
iter++  
until  $B$  is empty // i.e. no over  
used bins
```